

■ Python Operators

30 Interview Questions & Answers

Fresher-friendly • Speakable answers • Proper code indentation • Color-coded revision

How to study: Read the Interview Answer first → understand the Example → memorize the Interview Tip.

1. Explain the difference between the `is` operator and the `==` operator when comparing two mutable objects.

■ Interview Answer

`==` checks whether two objects have equal values. `is` checks whether both variables refer to the exact same object in memory. For mutable objects like lists, two separate lists can have the same contents but still be different objects.

■ Code Example

```
a = [1, 2]
b = [1, 2]
c = a

print(a == b) # True
print(a is b) # False
print(a is c) # True
```

■ Interview Tip

Remember: `==` → same value, `is` → same object. `is` is commonly used with `None`.

2. How does Python's `@` matrix multiplication operator work with custom classes? Provide an example implementation of `__matmul__`.

■ Interview Answer

The `@` operator is used for matrix multiplication. For a user-defined class, Python can call `__matmul__()` when the object appears on the left side of `@`.

■ Code Example

```
class Matrix:
    def __matmul__(self, other):
        return "matrix multiplication"

a = Matrix()
b = Matrix()

print(a @ b)
```

■ Interview Tip

`@` is different from `*`: `*` normally means multiplication, while `@` represents matrix multiplication.

3. Describe the short-circuit evaluation behavior of the and and or operators with side-effects.

■ Interview Answer

`and` and `or` use short-circuit evaluation. Python stops evaluating as soon as the final result is already known. With `and`, a false first operand is enough; with `or`, a true first operand is enough.

■ Code Example

```
def show():
    print("Function called")
    return True

print(False and show())
print(True or show())
```

■ Interview Tip

`and` stops at the first falsy value; `or` stops at the first truthy value.

4. What is the precedence order between bitwise operators (&, |, ^, ~, <<, >>) and logical operators (and, or, not)?

■ Interview Answer

Python gives bitwise operators higher precedence than logical `and` and `or` operators. The exact precedence among all operators matters, so parentheses are the safest way to make a complex expression clear.

■ Code Example

```
x = 4
y = 2

result = x | y and 3
print(result)
```

■ Interview Tip

When an expression is complex, use parentheses rather than relying on memorized precedence.

5. How can you overload the augmented assignment operators (e.g., +=, *=) in a user-defined class?

■ Interview Answer

A class can customize augmented assignment using methods such as `\_\_iadd\_\_()` for `+=` and `\_\_imul\_\_()` for `\*=`. These methods can modify the object in place or return another object.

■ Code Example

```
class Counter:
    def __init__(self, value):
        self.value = value

    def __iadd__(self, other):
        self.value += other
        return self

c = Counter(5)
c += 2

print(c.value)
```

■ Interview Tip

Key methods include `\_\_iadd\_\_`, `\_\_isub\_\_`, `\_\_imul\_\_`, and `\_\_itruediv\_\_`.

6. Discuss the role of the `__richcmp__` methods (`__lt__`, `__le__`, `__gt__`, `__ge__`, `__eq__`, `__ne__`) in customizing comparison operators. Why might you need to implement all six?

■ Interview Answer

Python lets a class define comparison behavior through separate special methods: `__lt__`, `__le__`, `__gt__`, `__ge__`, `__eq__`, and `__ne__`. Implement only what your class needs; all six may be useful when complete comparison behavior is required.

■ Code Example

```
class Student:
    def __init__(self, marks):
        self.marks = marks

    def __lt__(self, other):
        return self.marks < other.marks
```

■ Interview Tip

Python exposes individual rich-comparison methods; there is no normal Python `__richcmp__()` method to implement.

7. Explain why the expression `a < b < c` is evaluated as a chained comparison and not as `(a < b) < c`. What are the performance implications?

■ Interview Answer

`a < b < c` is a chained comparison. Python checks `a < b` and `b < c`, and the middle expression is evaluated only once. This is clearer and can avoid evaluating the middle expression twice.

■ Code Example

```
a = 1
b = 2
c = 3

print(a < b < c)  # True
```

■ Interview Tip

`a < b < c` means `a < b` and `b < c`.

8. In Python, the `in` operator can be used with many container types. How does it work under the hood for user-defined objects? Which special method must be implemented?

■ Interview Answer

For a user-defined container, the membership operator `in` can use the `__contains__()` special method. Implementing it gives the class direct control over membership testing.

■ Code Example

```
class Box:
    def __init__(self, items):
        self.items = items

    def __contains__(self, item):
        return item in self.items

box = Box([10, 20, 30])

print(20 in box)
```

■ Interview Tip

`in` → `__contains__()` is the key method to mention for a custom container.

9. Demonstrate how the `:=` (walrus) operator can be used inside a list comprehension that filters and transforms data simultaneously.

■ Interview Answer

The `:=` operator assigns a value inside an expression. In a comprehension, it can calculate a value once and then use it in the filtering condition or output.

■ Code Example

```
words = ["cat", "elephant", "dog"]  
  
result = [n for word in words if (n := len(word)) > 3]  
  
print(result) # [8]
```

■ Interview Tip

Use the walrus operator carefully; a normal assignment before a comprehension may be clearer.

10. Why does the expression `bool([]) + bool({})` return 0? Explain the underlying type coercion rules for arithmetic with booleans.

■ Interview Answer

An empty list and an empty dictionary are falsy, so both `bool()` calls return `False`. In Python, `bool` is a subclass of `int`: `False` behaves like `0` and `True` like `1` in arithmetic.

■ Code Example

```
print(bool([]))  
print(bool({}))  
print(bool([]) + bool({})) # 0
```

■ Interview Tip

`False` behaves like `0` and `True` like `1` in numeric contexts.

11. How does the // floor division operator behave with negative numbers? Provide examples that illustrate the difference from true division /.

■ Interview Answer

`/` performs true division. `//` performs floor division, which rounds the mathematical result down toward negative infinity, not toward zero.

■ Code Example

```
print(5 / 2)    # 2.5
print(5 // 2)   # 2
print(-5 / 2)   # -2.5
print(-5 // 2)  # -3
```

■ Interview Tip

The important example is `-5 // 2 == -3`, because floor means down toward negative infinity.

12. Describe how the * operator works for both sequence repetition and argument unpacking. What rules does Python follow to disambiguate the two uses?

■ Interview Answer

With a sequence and an integer, `\*` can repeat the sequence. In a function call, `\*` unpacks an iterable into positional arguments. Python determines the meaning from the syntax context.

■ Code Example

```
print([1, 2] * 2)

def add(a, b, c):
    return a + b + c

values = [10, 20, 30]
print(add(*values))
```

■ Interview Tip

`sequence \* number` → repetition; `\*iterable` in a call → positional unpacking.

13. Explain the effect of using the ~ (bitwise NOT) operator on a Python integer. How does it relate to two's complement representation?

■ Interview Answer

`~` performs bitwise inversion. A useful Python identity is `~x == -x - 1`. Python integers have arbitrary precision, so this identity is often the easiest way to reason about the result.

■ Code Example

```
print(~5)  # -6
print(~0)  # -1
```

■ Interview Tip

Easy formula: `~n = -n - 1`.

14. What is the result of 0 == False and 0 is False? Why does Python treat these cases differently?

■ Interview Answer

`0 == False` is `True` because equality compares their values and Boolean `False` has numeric value 0. `0 is False` is `False` because identity checks whether they are the same object.

■ Code Example

```
print(0 == False)  # True
print(0 is False)  # False
```

■ Interview Tip

Do not use `is` as a replacement for `==` when comparing ordinary values.

15. How can you implement a custom class that supports the @ operator for both left-hand and right-hand matrix multiplication? Which dunder methods are required?

■ Interview Answer

For left-hand `a @ b`, define `\_\_matmul\_\_()`. For reflected right-hand support, define `\_\_rmatmul\_\_()`.

■ Code Example

```
class Matrix:
    def __matmul__(self, other):
        return "left matrix multiply"

    def __rmatmul__(self, other):
        return "right matrix multiply"

m = Matrix()

print(m @ 5)
print(5 @ m)
```

■ Interview Tip

Key pair: `\_\_matmul\_\_()` and `\_\_rmatmul\_\_()`.

16. Discuss the potential pitfalls of using mutable default arguments with operators that modify the argument in place (e.g., list += other).

■ Interview Answer

Mutable default arguments are created once when the function is defined. If `+=` modifies that list in place, later calls can see changes made by earlier calls.

■ Code Example

```
def add_item(item, items=[]):
    items += [item]
    return items

print(add_item(1))
print(add_item(2))
```

■ Interview Tip

Best practice: use `None` as the default and create a new list inside the function.

17. How does the pow() built-in function differ from the ** exponentiation operator when a third modulus argument is supplied?

■ Interview Answer

`\*\*` performs exponentiation. `pow(base, exponent, modulus)` can perform modular exponentiation directly. The three-argument form is useful for large numbers because it avoids creating the full huge power first.

■ Code Example

```
print(2 ** 5)
print(pow(2, 5))
print(pow(2, 5, 3))
```

■ Interview Tip

The third modulus argument is supported by `pow()`, not by `\*\*`.

18. Why does the expression `5 / 2 == 5 // 2` evaluate to False in Python 3 but not in Python 2?

■ Interview Answer

In Python 3, `/` is true division, so `5 / 2` is `2.5`, while `5 // 2` is `2`. Therefore the comparison is false. Python 2 had different integer-division behavior for `/`.

■ Code Example

```
print(5 / 2)      # 2.5
print(5 // 2)     # 2
print(5 / 2 == 5 // 2) # False
```

■ Interview Tip

For modern interviews, focus on Python 3 behavior.

19. Show how to overload the `<<` and `>>` operators for a custom bit-vector class, and discuss any constraints imposed by Python's data model.

■ Interview Answer

A custom class can implement `__lshift__()` for `<<` and `__rshift__()` for `>>`. These methods define shifting behavior for the custom type.

■ Code Example

```
class Bits:
    def __init__(self, value):
        self.value = value

    def __lshift__(self, n):
        return Bits(self.value << n)

    def __rshift__(self, n):
        return Bits(self.value >> n)
```

■ Interview Tip

The operator methods should return an appropriate result and follow a consistent contract for the custom type.

20. How does the `or` operator determine the value it returns (as opposed to just a boolean)?

■ Interview Answer

`or` does not necessarily return `True` or `False`. It returns the first truthy operand. If the first operand is falsy, it returns the second operand.

■ Code Example

```
name = ""
result = name or "Guest"

print(result) # Guest

x = "Umesh"
print(x or "Guest") # Umesh
```

■ Interview Tip

`or` returns an operand, not necessarily a Boolean.

21. Explain the difference between `a & b` and `a and b` when `a` and `b` are sets. What does each expression evaluate to?

■ Interview Answer

For sets, `a & b` is set intersection and returns elements present in both sets. `a and b` is a logical operation based on truthiness; if `a` is truthy, it returns `b`.

■ Code Example

```
a = {1, 2, 3}
b = {2, 3, 4}

print(a & b)    # {2, 3}
print(a and b) # {2, 3, 4}
```

■ Interview Tip

`&` → set intersection; `and` → logical short-circuiting.

22. What is the purpose of the `__invert__` method, and how does it interact with the `~` operator for user-defined numeric types?

■ Interview Answer

`__invert__()` defines the behavior of the unary `~` operator for a user-defined class. When Python evaluates `~obj`, it can call `obj.__invert__()`.

■ Code Example

```
class Number:
    def __invert__(self):
        return "custom invert"

n = Number()

print(~n)
```

■ Interview Tip

Operator mapping: `~` → `__invert__()`.

23. Discuss how operator precedence changes when parentheses are introduced in a complex expression involving arithmetic, bitwise, and logical operators.

■ Interview Answer

Parentheses explicitly group an expression and can change the order in which operations are performed. They also make complex expressions easier to understand.

■ Code Example

```
x = 2 + 3 * 4
y = (2 + 3) * 4

print(x) # 14
print(y) # 20
```

■ Interview Tip

When several operator categories are mixed, parentheses improve clarity.

24. How can you use the `*` operator to unpack a dictionary into a function call (Python 3.5+), and what operator is used for merging dictionaries?

■ Interview Answer

For a function call, `*` unpacks an iterable into positional arguments, while `**` unpacks a dictionary into keyword arguments. Modern Python uses `|` for dictionary merging.

■ Code Example

```
def greet(name, city):
    print(name, city)

data = {"name": "Umesh", "city": "Indore"}

greet(**data)

a = {"x": 1}
b = {"y": 2}

print(a | b)
```

■ Interview Tip

Important: a dictionary is unpacked into keyword arguments with `**`, not `*`. Dictionary merging uses `|`.

25. Provide a scenario where using the `+=` operator on a tuple raises an error, and explain why.

■ Interview Answer

A tuple is immutable, so its elements cannot be changed in place. If a tuple contains a mutable object such as a list, `+=` can mutate that inner list and then fail when Python tries to assign the result back into the tuple slot.

■ Code Example

```
t = ([1, 2],)

try:
    t[0] += [3]
except TypeError as e:
    print(e)

print(t)
```

■ Interview Tip

Important interview point: tuples are immutable, but they can contain mutable objects.

26. Explain the behavior of the `==` operator when comparing a NumPy array with a scalar. How does this differ from Python's built-in list comparison?

■ Interview Answer

A NumPy array comparison with a scalar is element-wise, so `array == scalar` produces a Boolean array. A normal Python list compared with a scalar does not perform element-wise equality.

■ Code Example

```
# NumPy example:
# np.array([1, 2, 1]) == 1
# -> array([ True, False,  True])

print([1, 2, 1] == 1) # False
```

■ Interview Tip

Key phrase: NumPy arrays support vectorized element-wise comparisons.

27. How does Python's operator module expose the functionality of built-in operators? Give an example of using operator.attrgetter with the @ operator.

■ Interview Answer

The `operator` module provides function versions of many operators and attribute access operations. For example, `operator.add(a, b)` is similar to `a + b`, while `operator.attrgetter()` creates a callable that retrieves an attribute. The `@` operator is represented by `operator.matmul`.

■ Code Example

```
import operator

print(operator.add(2, 3))

get_name = operator.attrgetter("name")
```

■ Interview Tip

`attrgetter` is for attribute access; `operator.matmul` represents the `@` operation.

28. What is the result of `bool('False')` and why? Discuss the distinction between truthiness and boolean value of strings.

■ Interview Answer

`bool('False')` is `True` because the string is non-empty. Python does not interpret the text `'False'` as the Boolean value `False` when using `bool()`. An empty string is falsy; a non-empty string is truthy.

■ Code Example

```
print(bool("False")) # True
print(bool(""))      # False
```

■ Interview Tip

Common interview trick: `'False'` is a non-empty string, so it is truthy.

29. Show how to implement a class that supports the + operator for both left-hand and right-hand addition with built-in types (e.g., `int + MyNumber`). Which methods must be defined?

■ Interview Answer

Define `__add__()` for `my_number + 5`, and `__radd__()` for reflected addition such as `5 + my_number`.

■ Code Example

```
class MyNumber:
    def __init__(self, value):
        self.value = value

    def __add__(self, other):
        return self.value + other

    def __radd__(self, other):
        return other + self.value

n = MyNumber(10)

print(n + 5)
print(5 + n)
```

■ Interview Tip

Remember the pair: `__add__()` → normal addition; `__radd__()` → reflected addition.

30. Explain why the expression `1 < 2 < 3 > 2` is valid and what its final boolean result is. Detail the evaluation order and short-circuiting behavior.

■ Interview Answer

Python supports chained comparisons. This is evaluated like ``1 < 2 and 2 < 3 and 3 > 2``, with the middle values evaluated only as needed. All three comparisons are true, so the final result is ``True``.

■ Code Example

```
print(1 < 2 < 3 > 2) # True
```

■ Interview Tip

Think: ``1 < 2`` ■, ``2 < 3`` ■, ``3 > 2`` ■ → ``True``.